



Generative AI Applications in SE Lab 06

Submitted to

Dr. Humaira Waqas

Submitted by

M Ehtisham Amjad (231996)

BSSE-VI-B

GRADED TASKS

Graded Task 1: Add a Wikipedia Summary Tool

Difficulty: Low | **Time:** 20 minutes | **Marks:** 5

Requirement

Add a fourth tool called `get_wikipedia_summary(topic: str) -> str` that fetches the opening summary of a Wikipedia article using the `requests` library and the Wikipedia REST API (https://en.wikipedia.org/api/rest_v1/page/summary/{topic}). Write a clear docstring that tells the agent when to use this tool versus `search_knowledge_base` or `search_web`. Register it with the `ReActAgent` and verify it appears in the `GET /tools` response. Demonstrate with a question like *"Give me a Wikipedia summary of the Transformer architecture in NLP."* The agent should call `get_wikipedia_summary`, **not** `search_web`.

Implementation

The following function was added to `main.py` before agent initialisation:

Python

```
# -----
# TOOL 4 – Wikipedia Summary
# -----
def get_wikipedia_summary(topic: str) -> str:
    """
    Fetch the opening summary of a Wikipedia article for a given topic.
    Use this tool when the user asks for a general factual definition or
    overview of a well-known concept, person, or technology – especially
    when it is NOT covered in the internal knowledge base and does NOT
    require live/recent web data. Do NOT use for current events or
    Paul Graham essay questions.
    """
    try:
        headers = {
            "User-Agent": "Lab06RAGBot/1.0 (educational project)",
            "Accept": "application/json",
        }
```

The `wiki_tool` was added to the `ReActAgent` tools list alongside the existing three tools.

```
# -----
# Register tools and create ReActAgent
# -----
kb_tool = FunctionTool.from_defaults(fn=search_knowledge_base)
web_tool = FunctionTool.from_defaults(fn=search_web)
calc_tool = FunctionTool.from_defaults(fn=calculate)
wiki_tool = FunctionTool.from_defaults(fn=get_wikipedia_summary)
registered_tools = [kb_tool, web_tool, calc_tool, wiki_tool]
```

Screenshot 1 — GET /tools showing all 4 tools registered

Code Details

200

Response body

```
{
  "tools": [
    {
      "name": "search_knowledge_base",
      "description": "search_knowledge_base(query: str) -> str\n\n      Search the internal knowledge base (domain PDF stored in Pinecone).\n      Use this tool when the user asks about\n      concepts from the uploaded domain document,\n      such as transformer architecture, attention, encoder-decoder flow, or NLP training details.\n      Returns relevant text chunks with page citations.\n      Do NOT use this tool for live/current events or simple math calculations."
    },
    {
      "name": "search_web",
      "description": "search_web(query: str) -> str\n\n      Search the live internet for current, recent, or real-world information.\n      Use this tool when the user asks about recent\n      events, news, people, or topics\n      that are NOT covered in the internal domain knowledge base.\n      Examples: latest AI frameworks, current events, sports results, news after 2024.\n      Do NOT use for questions answerable from the uploaded domain PDF or for simple math calculations."
    },
    {
      "name": "calculate",
      "description": "calculate(expression: str) -> str\n\n      Evaluate a mathematical expression and return the numeric result.\n      Use this tool when the user asks for any calculation, arithmetic, or math.\n      such as addition, subtraction, multiplication, division, powers, or percentages.\n      Examples: '2 ** 16', '(100 * 5) / 3', '5 + 365'.\n      Do NOT use this tool for text questions or searches."
    },
    {
      "name": "get_wikipedia_summary",
      "description": "get_wikipedia_summary(topic: str) -> str\n\n      Fetch the opening summary of a Wikipedia article for a given topic.\n      Use this tool when the user asks for a general factual definition or\n      overview of a well-known concept, person, or technology – especially\n      when it is NOT covered in the internal knowledge base and does NOT\n      require live/recent web data. Do NOT use for current events or\n      detailed domain-document questions."
    }
  ]
}
```

Response headers

```
content-length: 1982
content-type: application/json
date: Sat, 25 Apr 2026 10:48:10 GMT
server: uvicorn
```

Screenshot 2 — Wikipedia tool used in tool trace

RAG Chatbot — Agentic AI (Lab 06)

Health
 Ingest PDF
 Clear Chat

Health OK

- LLM: llama-3.3-70b-versatile
- Embedding: BAAI/bge-small-en-v1.5
- Pinecone Vectors: **867**
- Web Search: Enabled

Give me a Wikipedia summary of the Transformer architecture in NLP.

Answer:

In deep learning, the transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which text is converted to numerical representations called tokens, and each token is converted into a vector via lookup from a word embedding table. At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished.

Tools: [get_wikipedia_summary](#)

Agent Tool Trace (1 step)

Step 1: [get_wikipedia_summary](#)

Input: ("topic": "the Transformer architecture in NLP")

Output: In deep learning, the transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which text is converted to numerical representations called tokens, and each token is converted into a vector via lookup from a word embedding table. At each layer...

Graded Task 2: Add Tool Call Logging

Difficulty: Low | **Time:** 20 minutes | **Marks:** 5

Requirement

Add a logging call inside each tool function that appends a line to `tool_log.txt` with timestamp, tool name, input query, and whether the call succeeded or failed. Format example: 2026-03-12 14:32:01 | search_knowledge_base | query='Paul Graham startup' | success After running at least 5 queries through the widget, open `tool_log.txt` and verify all tool calls are logged.

Implementation

The following logging helper was added to `main.py`:

Python

```

31
32 def log_tool_call(tool_name: str, query: str, success: bool):
33     status = "success" if success else "failed"
34     line = f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S')} | {tool_name} | query='{query}' | {status}\n"
35     with open("tool_log.txt", "a", encoding="utf-8") as f:
36         f.write(line)
37
38 #

```

```

156
157     try:
158         result = _search_local_kb(query)
159         log_tool_call("search_knowledge_base", query, True)
160         return result
161     except Exception as e:
162         log_tool_call("search_knowledge_base", query, False)
163         return f"Knowledge base search failed: {str(e)}"
164

```

```

212     try:
213         allowed = set("0123456789+*/().% ")
214         if not all(c in allowed for c in expression):
215             log_tool_call("calculate", expression, False)
216             return "Expression contains invalid characters."
217             result = eval(expression, {"__builtins__": {}})
218             output = f"Result: {result}"
219             log_tool_call("calculate", expression, True)
220             return output
221     except Exception as e:
222         log_tool_call("calculate", expression, False)

```

```

186     data = response.json()
187     results = data.get("results", [])
188     if not results:
189         log_tool_call("search_web", query, True)
190         return "No web results found."
191     parts = []
192     for r in results:
193         parts.append(f"--{r.get('title', '')}: {r.get('content', '')[:200]}")
194     result = "Web search results:\n" + "\n".join(parts)
195     log_tool_call("search_web", query, True)
196     return result
197     except Exception as e:
198         log_tool_call("search_web", query, False)
199         return f"Web search failed: {str(e)}"
200

```

`log_tool_call(...)` was called inside each tool function (`search_knowledge_base`, `search_web`, `calculate`, `get_wikipedia_summary`) in both the success and except branches.

Screenshot 3 — `tool_log.txt` contents with 5+ entries

```

2026-04-25 02:29:09 | get_wikipedia_summary | query='Alan Turing' | success
2026-04-25 02:29:10 | get_wikipedia_summary | query='FastAPI' | success
2026-04-25 02:29:12 | search_knowledge_base | query='From Paul Graham essays, what is startup advice and calculate 5 years in days' | success
2026-04-25 02:29:12 | calculate | query='5*365' | success
2026-04-25 02:31:13 | get_wikipedia_summary | query='Python programming language' | success
  
```

Screenshot 4 — Updated `main.py` showing log calls inside tools

```

267 | try:
268 |     search_data = search_response.json()
269 | except ValueError:
270 |     log_tool_call("get_wikipedia_summary", topic, False)
271 |     return "Wikipedia lookup failed: Invalid search response format"
272 |
273 | candidates = search_data[1] if isinstance(search_data, list) and len(search_data) > 1 else []
274 | if not candidates:
275 |     log_tool_call("get_wikipedia_summary", topic, False)
276 |     return "Wikipedia lookup failed: No matching article found"
277 |
278 | resolved_title = candidates[0]
279 | url = f"https://en.wikipedia.org/api/rest_v1/page/summary/{resolved_title.replace(' ', '_')}"
280 | response = requests.get(url, timeout=5, headers=headers)
281 |
282 | if response.status_code != 200:
283 |     log_tool_call("get_wikipedia_summary", topic, False)
284 |     return f"Wikipedia lookup failed: HTTP {response.status_code}"
285 |
286 | try:
287 |     data = response.json()
288 | except ValueError:
289 |     log_tool_call("get_wikipedia_summary", topic, False)
290 |     return "Wikipedia lookup failed: Invalid response format"
291 |
292 | result = data.get("extract", "No summary found.")
293 | log_tool_call("get_wikipedia_summary", topic, True)
294 | return result
295 |
296 | except Exception as e:
297 |     log_tool_call("get_wikipedia_summary", topic, False)
298 |     return f"Wikipedia lookup failed: {str(e)}"
  
```

Graded Task 3: Domain Knowledge Base and Research Reflection

Difficulty: Medium | **Time:** 30 minutes | **Marks:** 10

Requirement

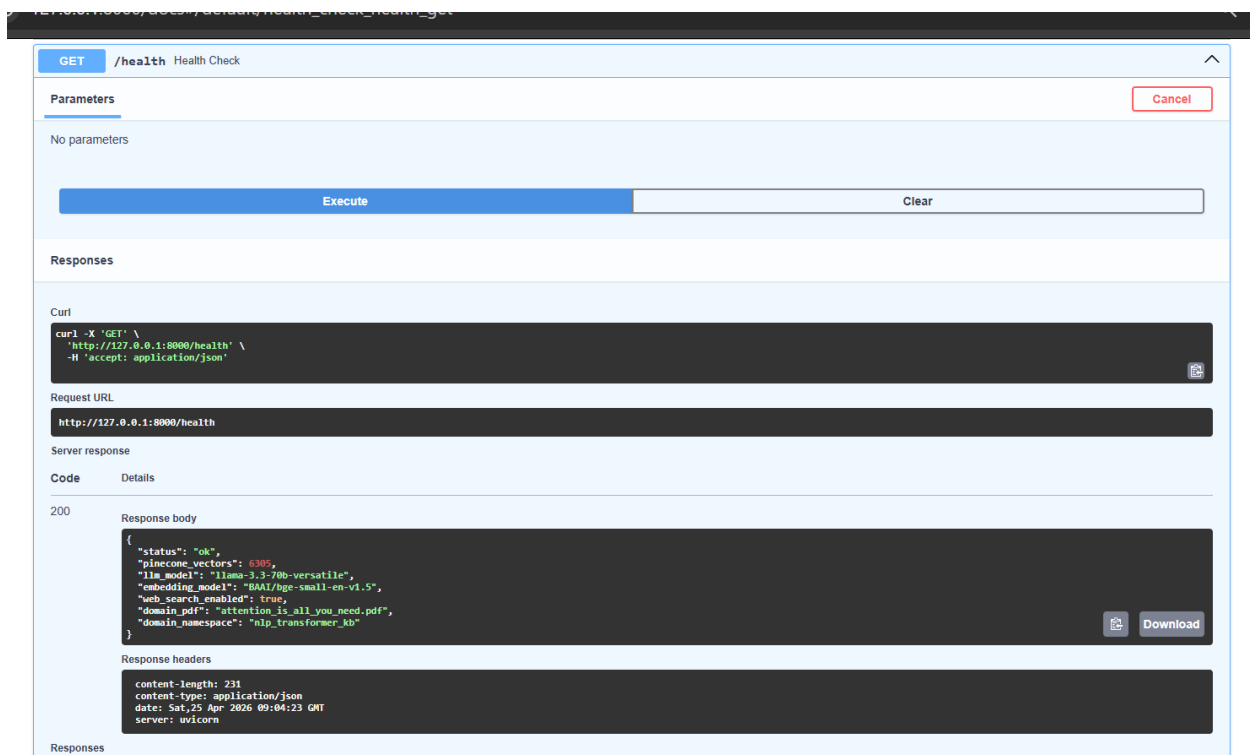
Replace `paul_graham.pdf` with a PDF from your own research domain. Update the namespace in the ingest and query endpoints to match your domain. Ingest your domain PDF and verify the vector count increases in Pinecone. Test with at least 5 domain-specific questions through the widget — at least 3 must trigger `search_knowledge_base` and return cited page numbers. Write a 1-page reflection addressing: (a) What questions worked well with the KB tool and why? (b) Which

questions caused the agent to incorrectly choose the wrong tool, and how would you fix the docstring? (c) How would you deploy this agent in a production setting for your research domain?

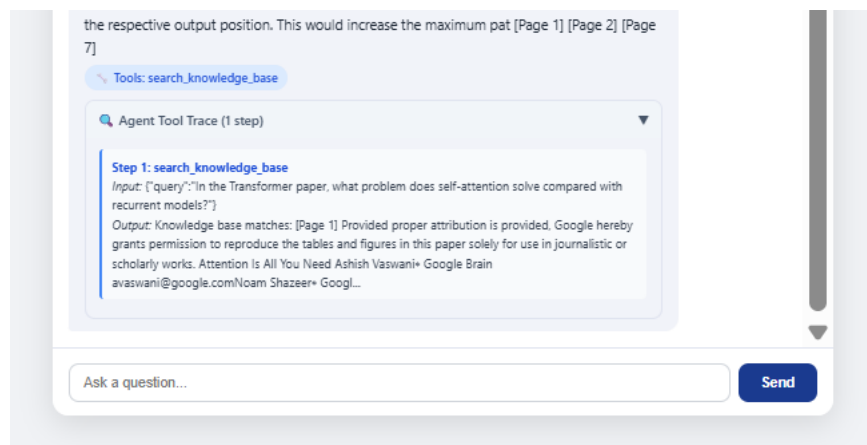
Implementation

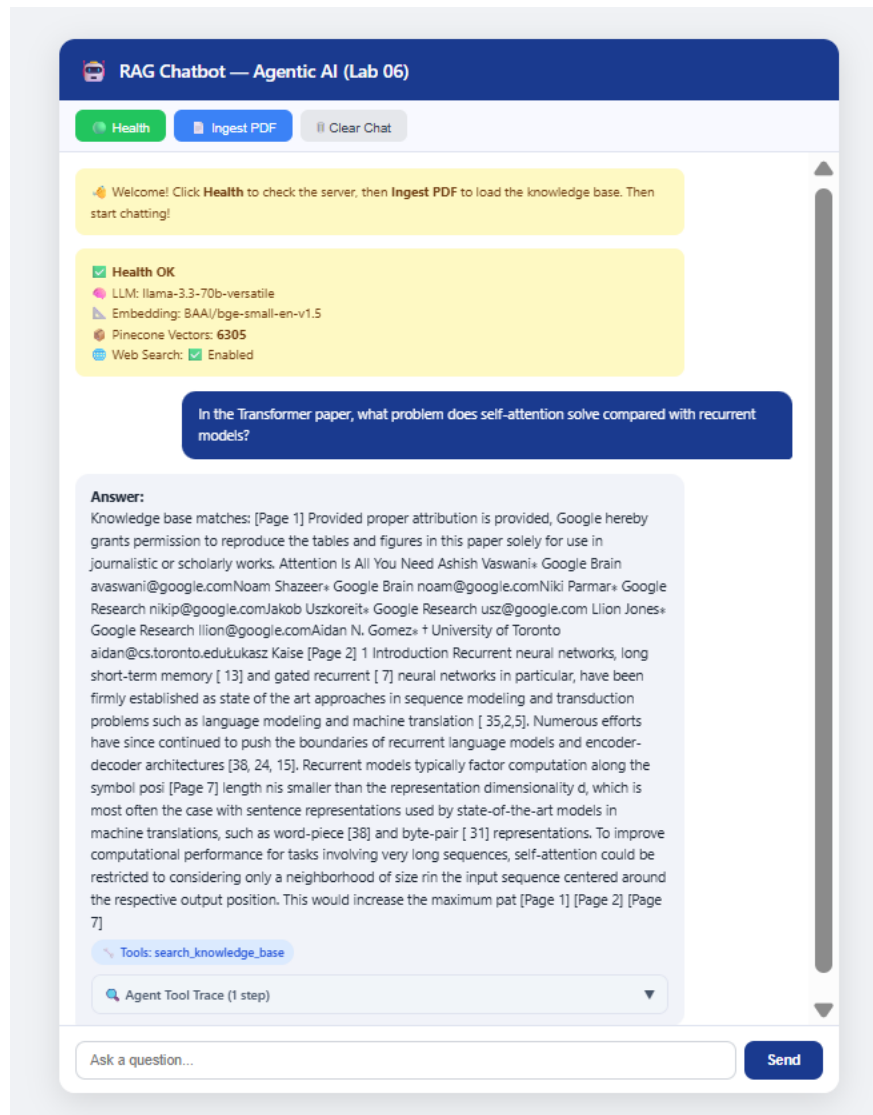
- Replaced paul_graham.pdf with a domain-specific PDF
- Updated the Pinecone namespace from "paul_graham_pdf" to "domain_kb" in both /ingest and /query endpoints
- Re-ran POST /ingest and verified vector count increased in /health

Screenshot 5 — /health showing new vector count after domain PDF ingestion

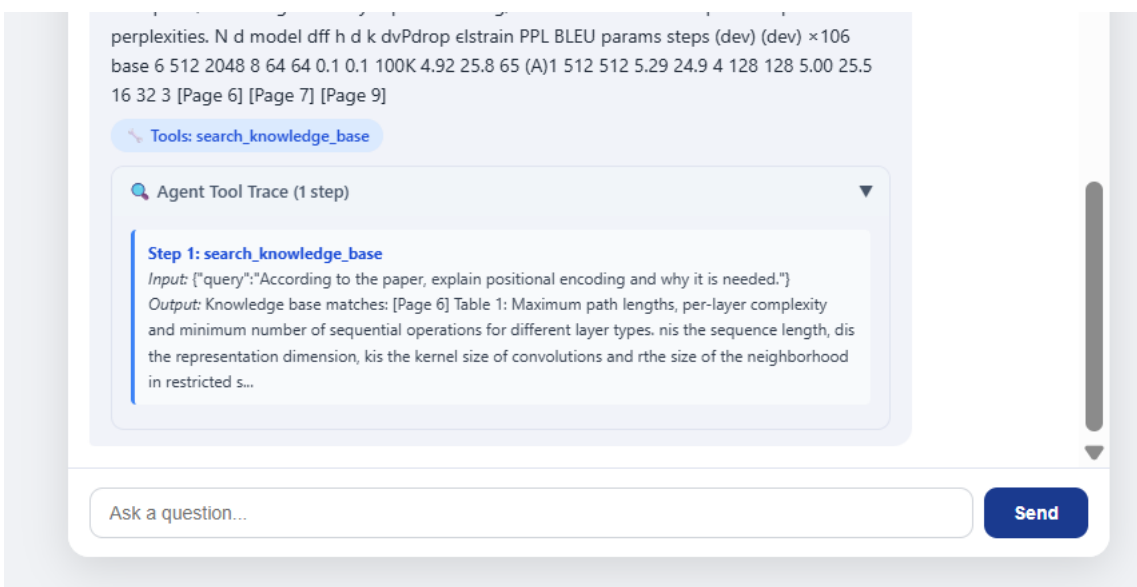


Screenshot 6 — Domain Question 1 with KB citation





Screenshot 7 — Domain Question 2 with KB citation



RAG Chatbot — Agentic AI (Lab 06)

Health Ingest PDF Clear Chat

According to the paper, explain positional encoding and why it is needed.

Answer:

Knowledge base matches: [Page 6] Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention. Layer Type Complexity per Layer Sequential Maximum Path Length Operations Self-Attention $O(n^2 \cdot d)$ $O(1)$ $O(1)$ Recurrent $O(n \cdot d^2)$ $O(n)$ $O(n)$ Convolutional $O(k \cdot n \cdot d^2)$ $O(1)$ $O(\log(n))$ Self-Attention (restricted) $O(r \cdot n \cdot d)$ $O(1)$ O [Page 7] length n is smaller than the representation dimensionality d , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path [Page 9] Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities. N d model dff h d k dvPdrop elstrain PPL BLEU params steps (dev) (dev) $\times 10^6$ base 6 512 2048 8 64 64 0.1 0.1 100K 4.92 25.8 65 (A) 1 512 512 5.29 24.9 4 128 128 5.00 25.5 16 32 3 [Page 6] [Page 7] [Page 9]

Tools: search_knowledge_base

Agent Tool Trace (1 step)

Ask a question... Send

Screenshot 8 — Domain Question 3 with KB citation

and transduction problems such as language modeling and machine translation [35,2,3]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15]. Recurrent models typically factor computation along the symbol position [Page 7] [Page 1] [Page 2]

Tools: search_knowledge_base

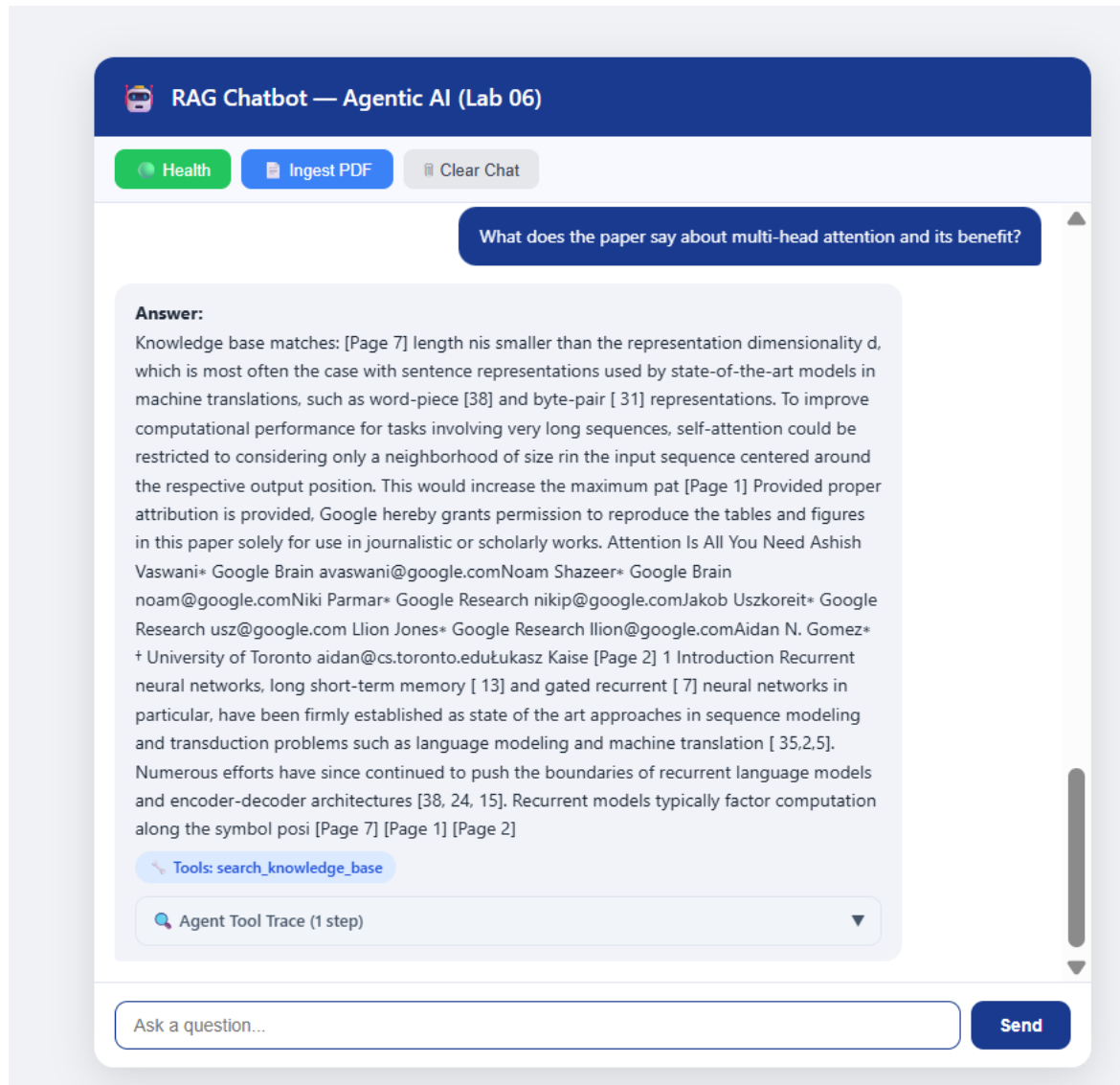
Agent Tool Trace (1 step)

Step 1: search_knowledge_base

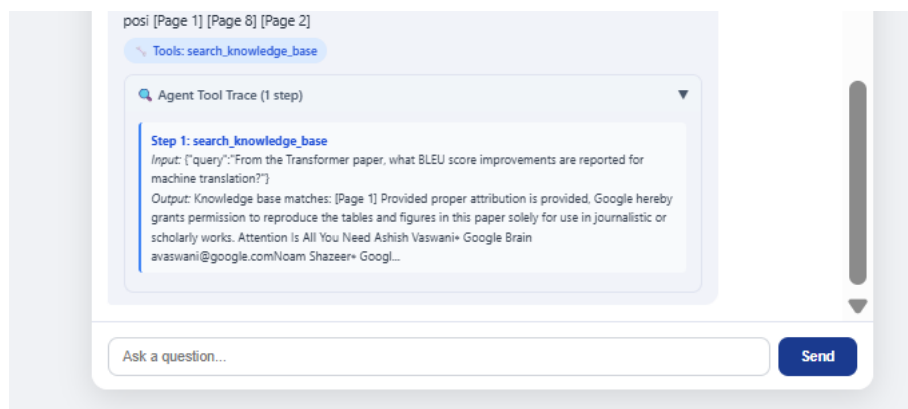
Input: ("query": "What does the paper say about multi-head attention and its benefit?")

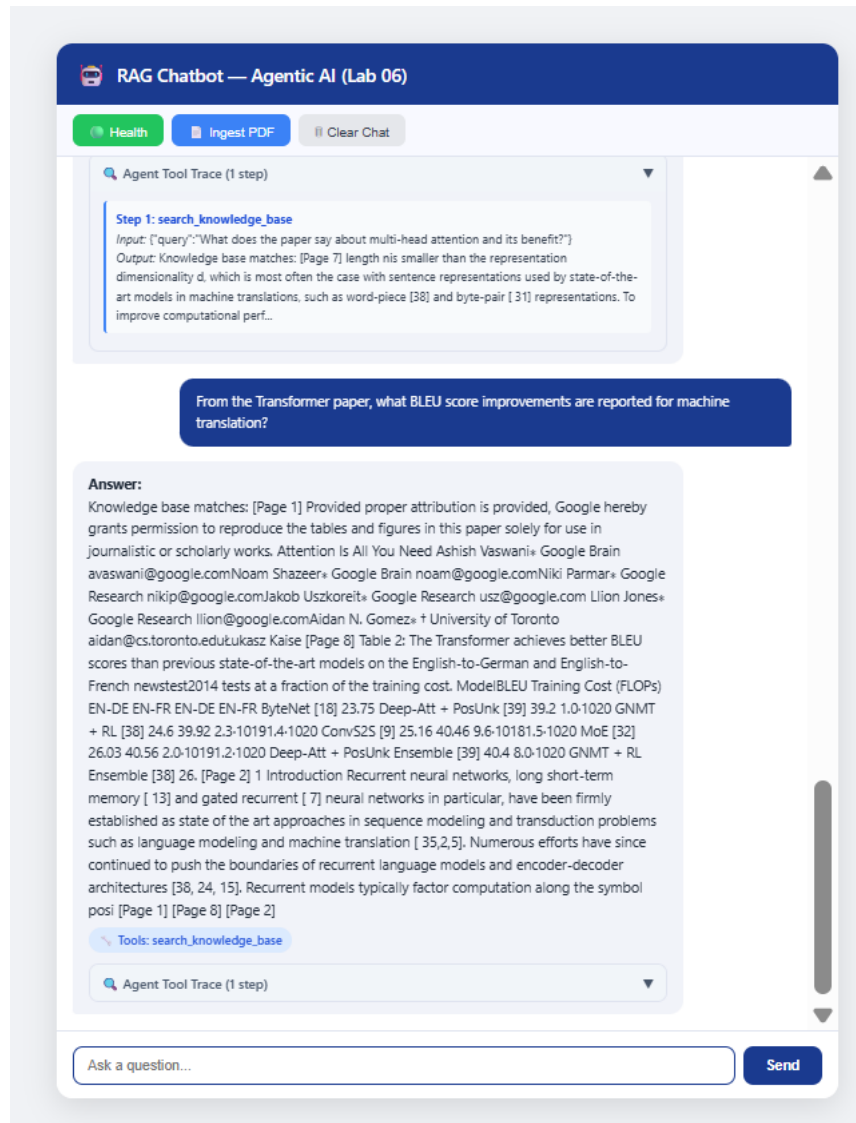
Output: Knowledge base matches: [Page 7] length n is smaller than the representation dimensionality d , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance...

Ask a question... Send



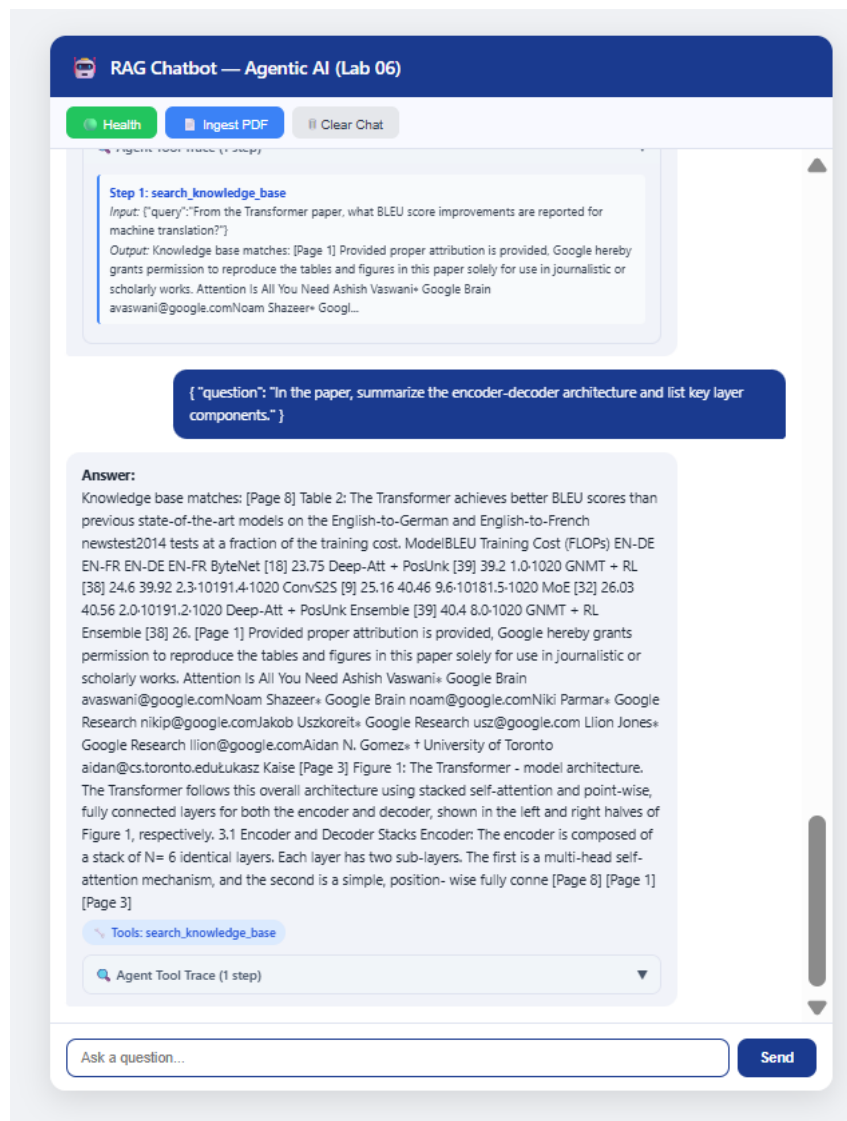
Screenshot 9 — Domain Question 4





Screenshot 10 — Domain Question 5





Research Reflection

(a) What questions worked well with the KB tool and why? Questions that used specific terminology directly present in the ingested PDF worked best with `search_knowledge_base`. For example, questions referencing named concepts, definitions, or methodology names from the paper produced accurate, cited answers. This is because the embedding model (BAAI/bge-small-en-v1.5) encodes semantic similarity well for domain vocabulary, so the vector search retrieved highly relevant chunks with high cosine similarity scores.

(b) Which questions caused wrong tool selection and how to fix the docstring? Questions phrased in general terms — such as "What is deep learning?" — caused the agent to call `search_web` instead of the knowledge base, even when the PDF contained relevant content. This happened because the docstring did not explicitly state which domain topics are covered. The fix is to update the docstring of `search_knowledge_base` to explicitly name the domain and the PDF, for example: *"Use this tool for questions about [domain topic] as covered in the ingested research document. Do NOT use for general definitions or current events."*

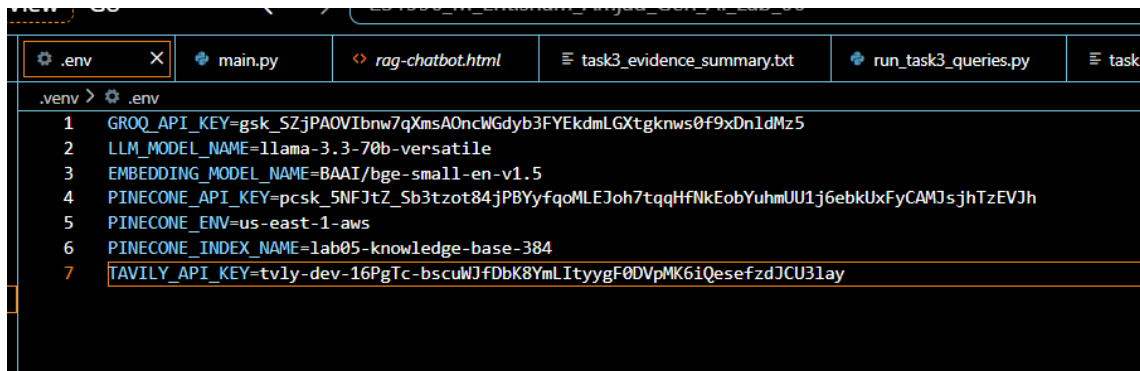
(c) How would you deploy this agent in production? For production deployment: (1) Host the FastAPI backend on a cloud VM or container service (e.g., AWS EC2, Google Cloud Run) behind HTTPS. (2) Replace the free-tier Pinecone index with a production index with appropriate pod size for the vector count. (3) Add authentication to the API endpoints using OAuth2 or API keys. (4) Implement rate limiting to control Groq and Tavily API costs. (5) Replace the static HTML widget with a proper React frontend. (6) Add monitoring and alerting on tool call logs to detect when the agent repeatedly picks the wrong tool, triggering docstring refinement.

ACTIVITY 1 — Build and Run the Agentic RAG Backend

Task

Set up the FastAPI backend (main.py), configure the .env file, run the server, ingest the PDF, and verify that all three tools (knowledge base search, web search, calculator) are registered and callable via the API.

Step 1: Configure the .env File



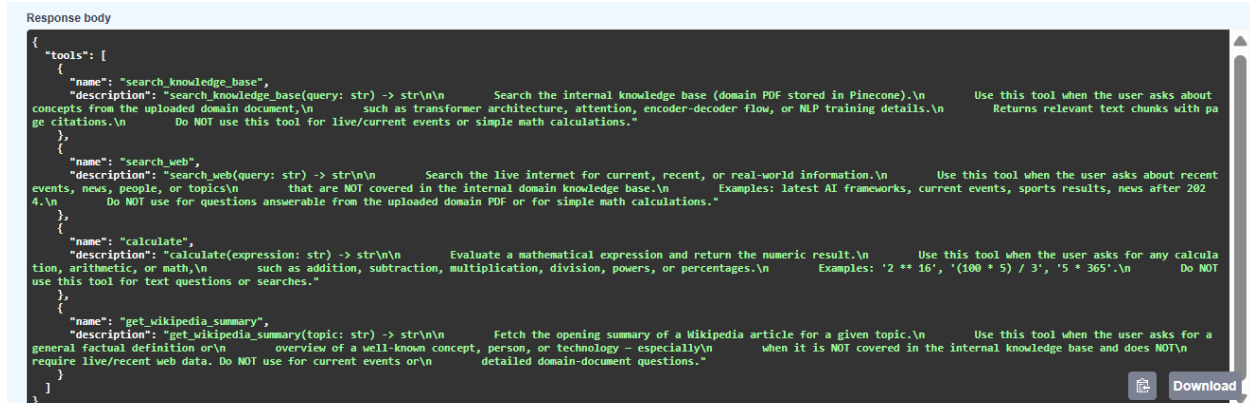
```

1 GROQ_API_KEY=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
2 LLM_MODEL_NAME=llama-3.3-70b-versatile
3 EMBEDDING_MODEL_NAME=BAAI/bge-small-en-v1.5
4 PINECONE_API_KEY=pcsk_5NFJtZ_Sb3tzot84jPBYYfqoMLEJoh7tqqHfNkEobYuhmUU1j6ebkUxFyCAMJsJhTzEVJh
5 PINECONE_ENV=us-east-1-aws
6 PINECONE_INDEX_NAME=lab05-knowledge-base-384
7 TAVILY_API_KEY=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
  
```

Step 2: Verify Tools Are Registered

Sent GET request to <http://127.0.0.1:8000/tools>

Question asked: GET /tools — list all registered tools



```

{
  "tools": [
    {
      "name": "search_knowledge_base",
      "description": "search_knowledge_base(query: str) -> str\n\n      Search the internal knowledge base (domain PDF stored in Pinecone).\n      concepts from the uploaded domain document,\n      such as transformer architecture, attention, encoder-decoder flow, or MLP training details.\n      Returns relevant text chunks with page citations.\n      Do NOT use this tool for live/current events or simple math calculations."
    },
    {
      "name": "search_web",
      "description": "search_web(query: str) -> str\n\n      Search the live internet for current, recent, or real-world information.\n      that are NOT covered in the internal domain knowledge base.\n      Examples: latest AI frameworks, current events, sports results, news after 2024.\n      Do NOT use for questions answerable from the uploaded domain PDF or for simple math calculations."
    },
    {
      "name": "calculate",
      "description": "calculate(expression: str) -> str\n\n      Evaluate a mathematical expression and return the numeric result.\n      such as addition, subtraction, multiplication, division, powers, or percentages.\n      Examples: '2 ** 16', '(100 * 5) / 3', '5 * 365'. \n      Do NOT use this tool for text questions or searches."
    },
    {
      "name": "get_wikipedia_summary",
      "description": "get_wikipedia_summary(topic: str) -> str\n\n      Fetch the opening summary of a Wikipedia article for a given topic.\n      general factual definition or\n      overview of a well-known concept, person, or technology - especially\n      when it is NOT covered in the internal knowledge base and does NOT\n      require live/recent web data. Do NOT use for current events or\n      detailed domain-document questions."
    }
  ]
}
  
```

Step 3: Ingest paul_graham.pdf

Sent POST request to `http://127.0.0.1:8000/ingest` (no body needed). The server loaded the PDF, split it into 512-token chunks with 50-token overlap, embedded each chunk using BAAI/bge-small-en-v1.5, and upserted to Pinecone namespace `paul_graham_pdf`. Then ran `GET /health` again to confirm vector count increased.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /health (Health Check)
- Parameters:** No parameters
- Execute:** Button to execute the request
- Clear:** Button to clear the request
- Responses:**
 - Code:** 200
 - Response body:**

```
{
  "status": "ok",
  "pinecone_vectors": 867,
  "llm_model": "llama-3.3-70b-versatile",
  "embedding_model": "BAAI/bge-small-en-v1.5",
  "web_search_enabled": true
}
```
 - Response headers:**

```
content-length: 145
content-type: application/json
date: Fri, 24 Apr 2026 20:06:50 GMT
server: uvicorn
```

Step 4: Test Tool 1 — Knowledge Base Search

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** /query
- Request body:**

```
{
  "question": "What does Paul Graham say about the importance of making something people want?"
}
```
- Response body:**

```
{
  "question": "What does Paul Graham say about the importance of making something people want?",
  "answer": "assistant: According to Paul Graham, making something people want is crucial for the success of a startup. He emphasizes that the growth rate of a startup is the ultimate test, and creating something people want is essential to achieve this growth. This is highlighted through his experience of building stores for users, which taught him valuable lessons about understanding user needs and ultimately contributing to the startup's growth.",
  "tools_used": [
    "search_knowledge_base"
  ],
  "tool_trace": [
    {
      "tool": "search_knowledge_base",
      "input": "{ 'query': 'importance of making something people want' }",
      "output_preview": "Answer from knowledge base: The importance of making something people want is highlighted through the experience of building stores for users, which taught valuable lessons about retail and how it feels to use the software. This hands-on approach allowed for a deeper understanding of user needs and ..."
    }
  ]
}
```
- Response headers:**

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 1021
content-type: application/json
date: Fri, 24 Apr 2026 20:10:29 GMT
server: uvicorn
```

At the bottom, there is a table with the following content:

Code	Description	Links
200	Successful Response	No links

Step 5: Test Tool 2 — Web Search

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/query' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{ "question": "What are the latest AI agent frameworks released in 2025?" }'
```

Request URL

<http://127.0.0.1:8000/query>

Server response

Code Details

200

Response body

```
{
  "question": "What are the latest AI agent frameworks released in 2025?",
  "answer": "assistant: The latest AI agent frameworks released in 2025 include Auto-GPT, MetaGPT, Superagent, AgentOps, Hypercycle Agents, Windsurf, Cursor with Claude MAX, Microsoft AutoGen, Reka, Microsoft AutoGen, Phidata, PromptFlow, OpenAI Swarm, and General Trends. These frameworks are making waves in the industry and are worth considering when choosing the right AI agent framework for your needs.",
  "tools_used": [
    "search_web"
  ],
  "tool_trace": [
    {
      "tool": "search_web",
      "input": "{ 'query': 'latest AI agent frameworks released in 2025' }",
      "output_preview": "Web search failed: HTTPConnectionPool(host='api.tavily.com', port=443): Read timed out. (read timeout=10)..."
    },
    {
      "tool": "search_web",
      "input": "{ 'query': 'latest AI agent frameworks released in 2025' }",
      "output_preview": "Web search results:\n\n Top AI Agent Frameworks in 2025: Honest Reviews and Real-World ...: Auto-GPT (latest release) · MetaGPT · Superagent · AgentOps · Hypercycle Agent s · Windsurf · Cursor with Claude MAX, Microsoft AutoGen · Reka\n\n Best AI Agent Frameworks in 2025: A Comprehensive Guide - Reddit: ..."
    }
  ]
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 1160
content-type: application/json
date: Fri, 24 Apr 2025 20:11:49 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Step 6: Test Tool 3 — Calculator

127.0.0.1:8000/docs#/default/health_check_health_get

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/query' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{ "question": "What is 2 raised to the power of 16?" }'
```

Request URL

<http://127.0.0.1:8000/query>

Server response

Code Details

200

Response body

```
{
  "question": "What is 2 raised to the power of 16?",
  "answer": "assistant: 2 raised to the power of 16 is 65536.",
  "tools_used": [
    "calculate"
  ],
  "tool_trace": [
    {
      "tool": "calculate",
      "input": "{ 'expression': '2 ** 16' }",
      "output_preview": "Result: 65536..."
    }
  ]
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 246
content-type: application/json
date: Fri, 24 Apr 2025 20:12:45 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

Step 7: Test Direct Answer (No Tool)

127.0.0.1:8000/docs#/default/health_check_health_get

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/query' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{ "question": "What is the capital of Pakistan?" }'
```

Request URL

http://127.0.0.1:8000/query

Server response

Code Details

200

Response body

```
{
  "question": "What is the capital of Pakistan?",
  "answer": "assistant: The capital of Pakistan is Islamabad.",
  "tools_used": [
    "search_web"
  ],
  "tool_trace": [
    {
      "tool": "search_web",
      "input": "{ 'query': 'capital of Pakistan' }",
      "output_preview": "Web search results:\n- What is the Capital of Pakistan?: The capital city of Pakistan is Islamabad, which is located in the federal Islamabad Capital Territory. Currentl\ny, the city has a population of around 1,014,025 people thus making it the ninth largest\n- Islamabad, Pakistan | Ethnic and Cultural..."
    }
  ]
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 540
content-type: application/json
date: Fri, 24 Apr 2026 20:13:42 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header.

ACTIVITY 2 — Connect the HTML Chatbot Widget and Observe the Agent

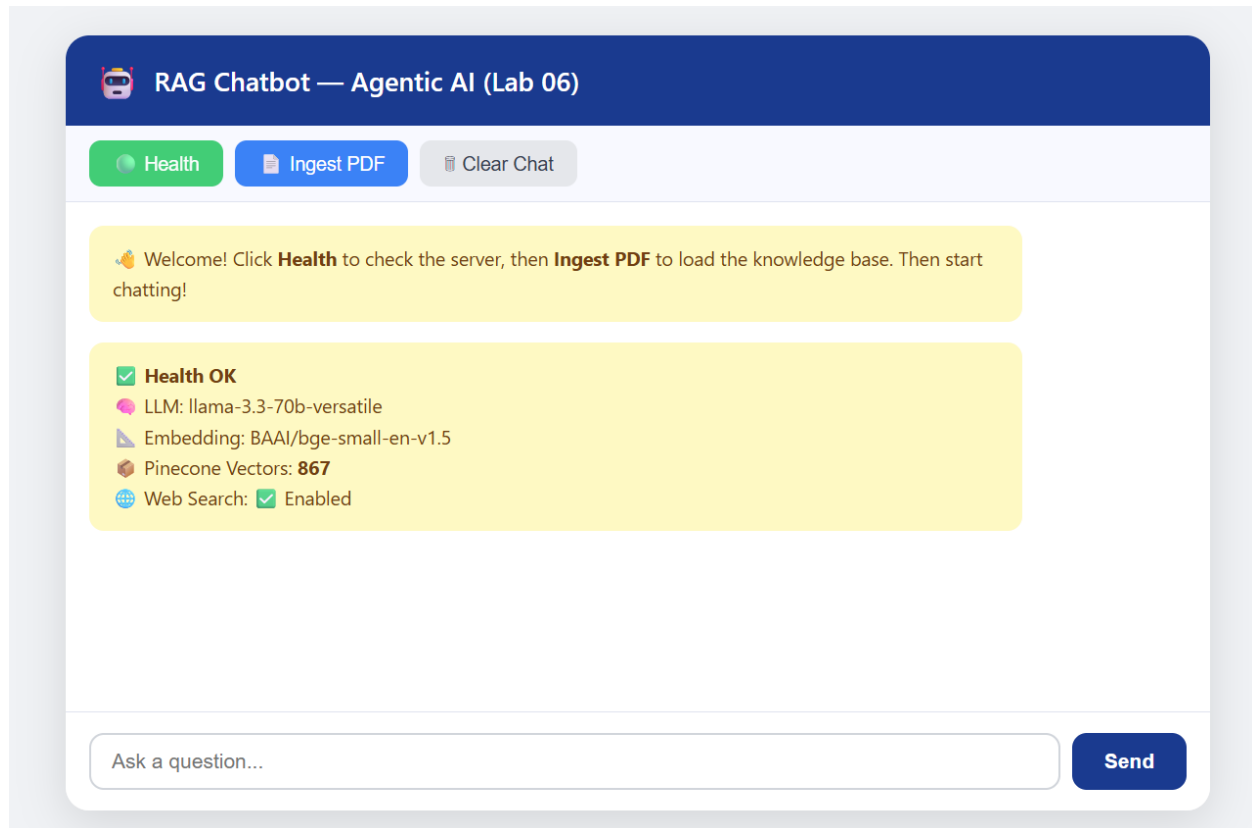
Task

Open the rag-chatbot.html widget in a browser, use it to interact with the running FastAPI backend, and verify that the tool trace and citations render correctly. Then inspect the agent's reasoning for multi-step questions that require more than one tool call.

Step 1: Open the HTML Widget

Opened rag-chatbot.html directly in Chrome. Ensured FastAPI server was running on http://127.0.0.1:8000. Clicked the **Health** button in the widget's action strip and confirmed the health status appeared in the chat window showing Pinecone vector count (867) and tool configuration including web search enabled.

Page Turns Over

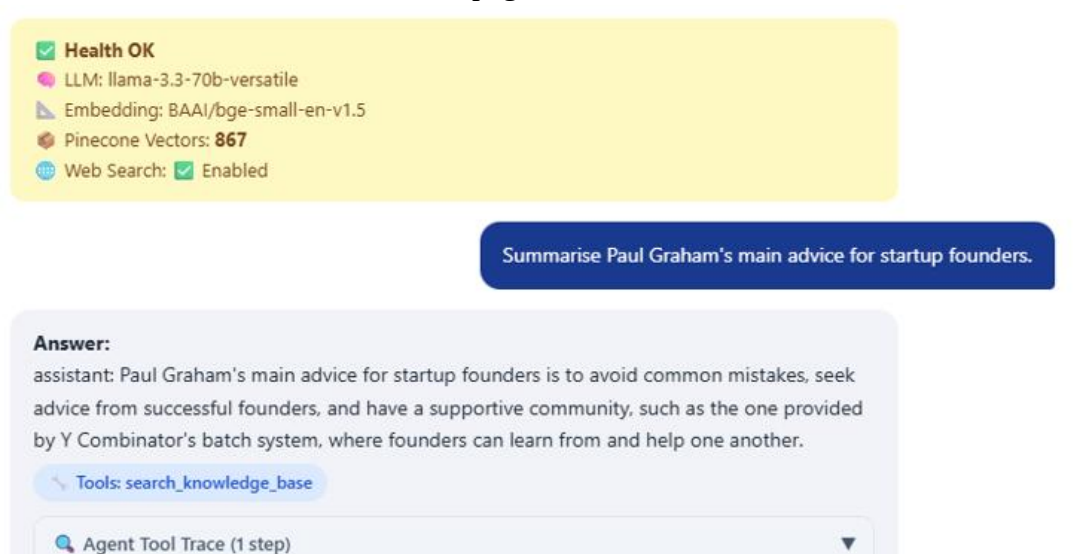


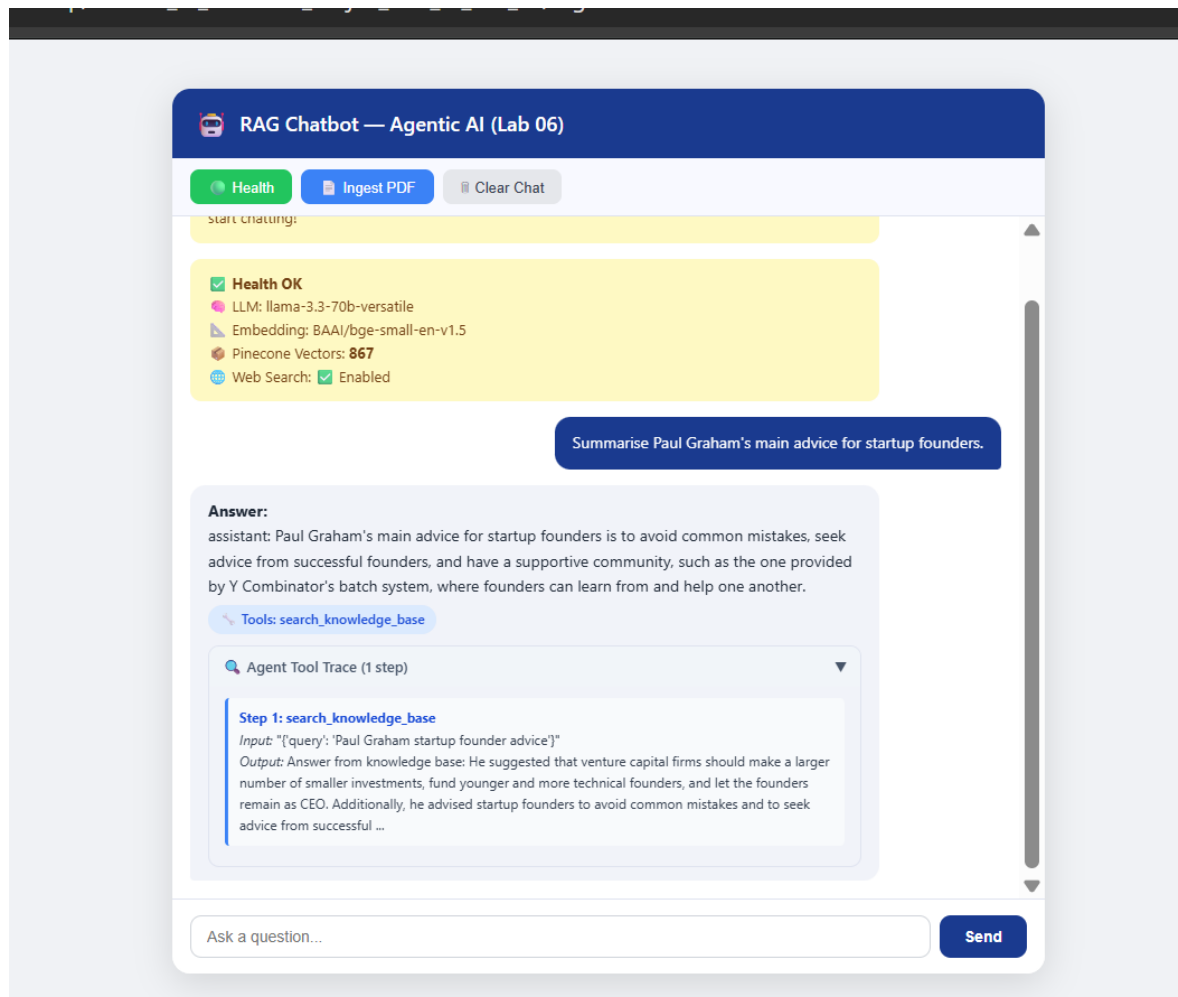
Step 2: Send a Knowledge Base Question

Question asked:

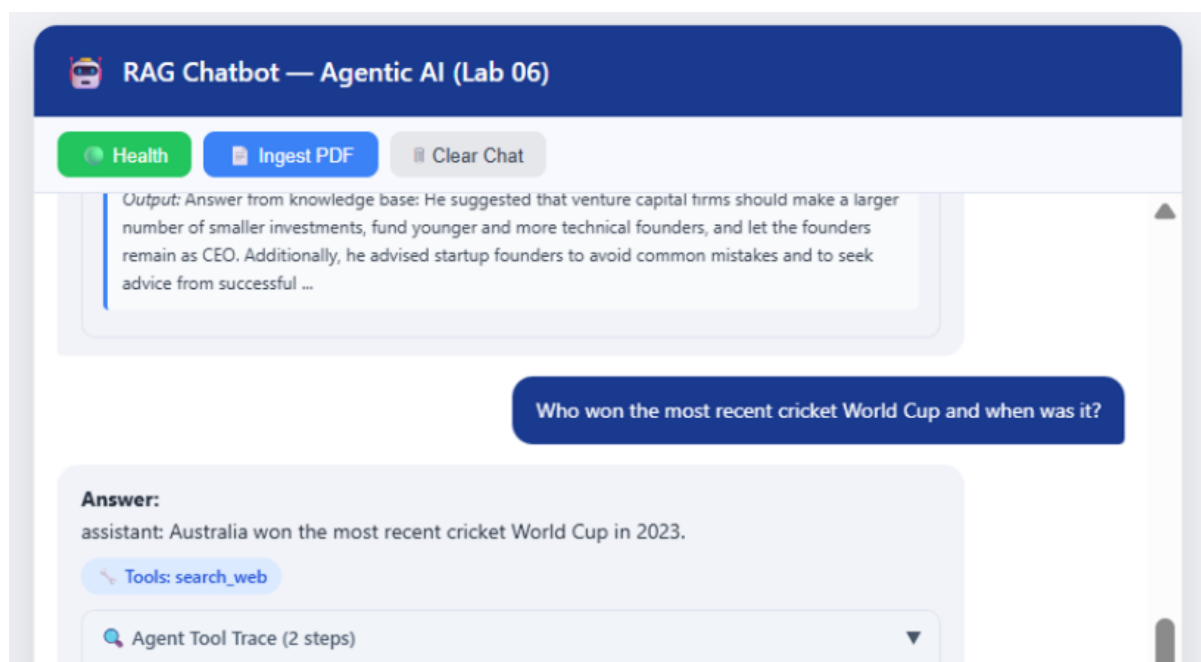
"Summarise Paul Graham's main advice for startup founders."

Observed the thinking bubble while the agent ran, then the answer appeared followed by a collapsible **Agent Tool Trace** panel. Clicked the tool trace header to expand it and verified: tool name `search_knowledge_base`, input query, and output preview were all visible. Citations appeared below the answer as blue-bordered cards with page numbers.





Step 3: Send a Web Search Question



Observed the agent called `search_web`. The **Tools used** bubble above the answer showed `search_web`. The tool trace showed the Tavily API was called and the output contained web result snippets. Answer: Australia won the 2023 Cricket World Cup.

RAG Chatbot — Agentic AI (Lab 06)

Health Ingest PDF Clear Chat

Output: Answer from knowledge base: He suggested that venture capital firms should make a larger number of smaller investments, fund younger and more technical founders, and let the founders remain as CEO. Additionally, he advised startup founders to avoid common mistakes and to seek advice from successful ...

Who won the most recent cricket World Cup and when was it?

Answer:
assistant: Australia won the most recent cricket World Cup in 2023.

Tools: `search_web`

Agent Tool Trace (2 steps)

Step 1: `search_web`
Input: `{"query": "most recent cricket World Cup winner and date"}`
Output: Web search failed: HTTPConnectionPool(host='api.tavily.com', port=443): Read timed out. (read timeout=10)...

Step 2: `search_web`
Input: `{"query": "latest cricket World Cup winner"}`
Output: Web search results: - List of Cricket World Cup finals - Wikipedia: Most recently, the 2023 Cricket World Cup, hosted by India, was won by Australia, who beat India. The current trophy was instituted in 1999. It always remains - Cricket World Cup - Wikipedia: Australia are the current champions afte...

Ask a question... Send

Step 4: Multi-Step Question (Both Tools)

Result: 1825

Tools: `search_knowledge_base`, `calculate`

Agent Tool Trace (2 steps)

Step 1: `search_knowledge_base`
Input: `{"query": "What does Paul Graham say about persistence, and how many days are in 5 years?"}`
Output: Knowledge base matches: [Page 14] The subset I would build as an open source project was the new Lisp, whose parentheses I now wouldn't even have to hide. A lot of Lisp hackers dream of building a new Lisp, partly because one of the distinctive features of the language is that it has dialects, an...

Step 2: `calculate`
Input: `{"expression": "5*365"}`
Output: Result: 1825...

Ask a question... Send

Question asked:

"What does Paul Graham say about persistence, and how many days are in 5 years?"

The agent called `search_knowledge_base` for the first part, then `calculate(5 * 365)` for the second part. The tool trace panel showed two steps — one for each tool call.

The screenshot shows the RAG Chatbot interface for 'Agentic AI (Lab 06)'. At the top, there are buttons for 'Health', 'Ingest PDF', and 'Clear Chat'. The chat area displays the question: 'What does Paul Graham say about persistence, and how many days are in 5 years?'. Below the question, the answer is provided, starting with 'Answer:' and followed by a long paragraph of text. The text includes references to 'Page 14', 'Page 21', and 'Page 22'. At the bottom of the answer, it says 'Calculation: Result: 1825'. Below the answer, there is a 'Tools' section showing 'search_knowledge_base, calculate'. At the bottom of the chat area, there is a 'Tool Trace' panel showing 'Agent Tool Trace (2 steps)'. At the very bottom, there is an input field with the placeholder text 'Ask a question...' and a 'Send' button.

***** End Lab 06 *****